

Санкт-Петербургский государственный университет
Факультет прикладной математики – процессов управления
ООП СВ.5005.2022 Прикладная математика, фундаментальная информатика и программирование

Шаго Павел Евгеньевич

*Математическое моделирование и реализация планировщика
процессов для гетерогенных процессорных архитектур*

Выпускная квалификационная работа

Научный руководитель:
кандидат физ.-мат. наук, доцент
Корхов В. В.

Рецензент:
эксперт по архитектуре и сетевому стеку,
ООО НИЦ АЭРОДИСК, Анохин Ю. В.

Санкт-Петербург
2026

Цель работы

разработка и экспериментальная оценка планировщика, обеспечивающего более высокую эффективность вычислений на гетерогенном оборудовании по сравнению со штатным планировщиком ядра Linux

Предмет исследования

методы планирования, учитывающие различие вычислительной мощности ядер

Гетерогенные процессоры в современных устройствах

Сегмент	Примеры	Массово	Сегодня
Смартфоны	Apple A, Snapdragon, Dimensity, Exynos, Tensor	2013	все актуальные
Ноутбуки	Apple M, Intel Core Ultra, Ryzen AI, Snapdragon X	2020	все актуальные
Настольные ПК	Intel Core 12+, Intel Core Ultra	2021	частично
Серверы	NVIDIA GB10	—	экспериментально

Тренд: гетерогенные P/E-архитектуры, возникшие в мобильных SoC, стали нормой для процессоров ПК и ноутбуков. Серверные CPU пока остаются однородными

- 1 Проанализировать и определить наиболее перспективный подход к реализации планировщика
- 2 Построить математическую модель, учитывающую различную вычислительную мощность ядер процессора
- 3 Реализовать разработанный алгоритм и собрать воспроизводимый набор бенчмарков
- 4 Провести эксперимент на гетерогенном оборудовании и оценить результаты в сравнении с базовым планировщиком Linux EEVDF

Сравнение подходов к реализации планировщика

Критерий	В ядре	Модуль	gh0St	sched_ext
Без пересборки ядра	—	+	±	+
Устойчивость к ошибкам реализации	—	—	+	+
Низкие накладные расходы	+	+	—	±
Поддерживаемый интерфейс замены	—	—	±	+
Переносимость между версиями ядра	—	—	±	+

Вывод: sched_ext является лучшим вариантом по приведённым критериям. Умеренные накладные расходы фреймворка — сознательный компромисс за остальные преимущества

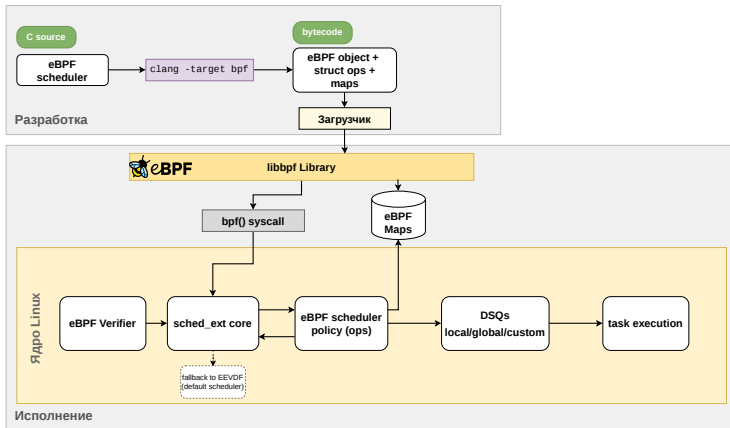


Рис. 1. Механизм работы sched_ext

DSQ (dispatch queue)

- SCX_DSQ_LOCAL
- SCX_DSQ_GLOBAL
- пользовательские DSQ

Цикл планирования

- select_cpu()
- enqueue()
- dispatch()
- running / stopping

Загрузка планировщика

- компиляция
- вызов bpf()
- верификатор ядра

Аппаратная платформа

- Ядра: $\mathcal{P} = \{P_1, \dots, P_{K_P}\}$,
 $\mathcal{E} = \{E_1, \dots, E_{K_E}\}$
- Дискретное время $t = 1, 2, 3, \dots$
- Замедление на E-ядре: $\sigma = \eta_P / \eta_E > 1$
- Общий ISA (любая задача может быть выполнена на любом ядре)

Задачи как агенты

- Тип $\theta_i = (v_i, l_i)$: ценность v_i , длина l_i
- Полезность как маргинальный вклад в социальное благосостояние W :
 $U_i = W(s^t) - W_{-i}(s^t) \geq 0$
- Бюджет $B_i(\chi_i) > 0$ по классу обслуживания χ_i :

реального времени
интерактивный
пакетный
фоновый

Временные кванты P/E-ядер процессора – **недолговечный ресурс** (неиспользованный квант теряется безвозвратно). Их распределение моделируется динамическим аукционом VCG с задачами-агентами, максимизирующим социальное благосостояние

Состояние $s^t = (\mathbf{x}^t, \mathbf{b}^t, \Theta^t, \mathcal{N}^t)$

- $\mathbf{x}^t$ – остаточные длительности контрактов
- $\mathbf{b}^t$ – остаточные бюджеты
- Θ^t – типы задач, $\mathcal{N}^t$ – активные задачи

Действие $a_i^t = (a_{i,P}, a_{i,E}) \in \{0, 1\}^2$,
 $a_{i,P} + a_{i,E} \leq 1$

$$\sum_i a_{i,P} \leq K_P^t, \quad \sum_i a_{i,E} \leq K_E^t$$

(K_κ^t – число свободных ядер кластера κ в момент t)

Эффективная ценность ($\kappa \in \{P, E\}$)

$$\phi_\kappa(\theta_i) = v_i - c_\kappa m_\kappa(l_i)$$

$$m_P(l) = l, \quad m_E(l) = \lceil l\sigma \rceil$$

Социальное благосостояние (Беллман, $\delta \in (0, 1)$):

$$W(s^t) = \max_{a^t} \left[\sum_i \phi_i^a + \delta \mathbb{E}[W(s^{t+1})] \right]$$

$$\phi_i^a = a_{i,P} \phi_P(\theta_i) + a_{i,E} \phi_E(\theta_i)$$

Платёж VCG

$$p_i^t = W_{-i}(s^t) - (W(s^t) - \phi_\kappa(\theta_i))$$

Решение аукциона

Задача направляется в DSQ_P или DSQ_E по $\kappa^* = \arg \max_{\kappa \in \{P, E\}} \phi_{\kappa}(\theta_i)$

DSQ_P

$$\phi_P \geq \phi_E$$

P-ядро выгоднее по
эффективной ценности

DSQ_E

$\phi_E > \phi_P$ либо
короткая задача при
перегрузке P-кластера

$$Q_P \geq K_P \text{ и } \\ Q_P K_E > Q_E K_P$$

DSQ_Starved

$$B_i^t < B_i^{\max}/10$$

Низкий приоритет,
обслуживается
после пополнения

Ослабления модели в реализации

- v_i сообщается через вес задачи $p \rightarrow \text{scx.weight}$
- $W_{-i}(s^t) \rightarrow$ кластерная EWMA $\bar{W}_{\kappa}$ по реализованным $|\phi|$
- $l_i \rightarrow$ EWMA по истории исполнения
- Контракт – горизонт планирования, не атомарная единица
- Бюджет – корзина токенов с пополнением во сне ($B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$, $\Delta B_i \propto w_i$)

Система

Ядро Linux 7.0,
Arch Linux
CPU: Intel Core
Ultra 7 265K
8P + 12E, без HT

Сравнение

Linux EEVDF
(эталон)
scx_EEVDF
scx_LAVD
scx_A1349 (VCG)

Бенчмарки

- **hackbench** – массовое порождение процессов с интенсивным межпроцессным обменом и постоянными переключениями контекста
- **sysbench** – `oltp_read_only`, транзакции чтения к PostgreSQL. Потoki блокируются на ожидании ответа базы данных
- **schbench** – имитация веб-обработчика, рабочие потоки чередуют короткие фазы ожидания и матричных вычислений

Метрики

hackbench: время (с)
sysbench: TPS
schbench: RPS, p99 / p99.9 (мкс)
RAPL package: энергия (Дж)

Прогоны

3 уровня нагрузки
 $N = 6$, 95% ДИ

Hackbench: время (с) (меньше — лучше)

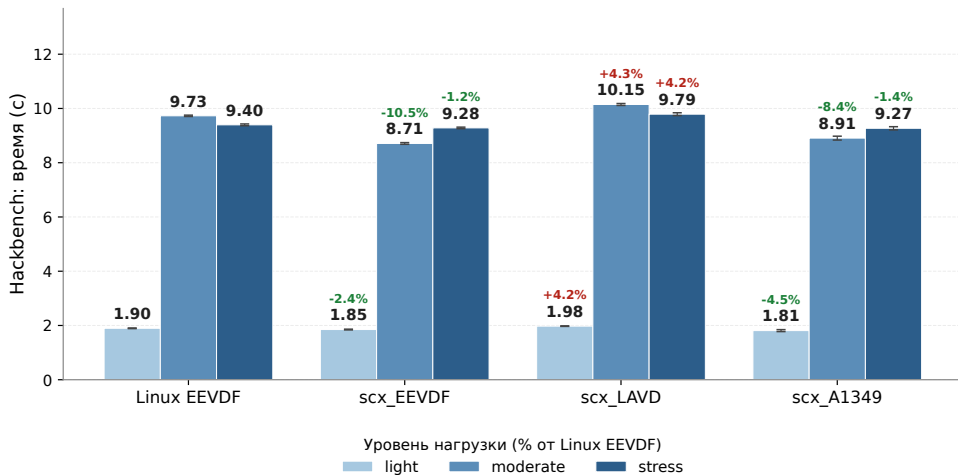


Рис. 2. Результаты hackbench

Sysbench OLTP, транзакций в секунду (больше — лучше)

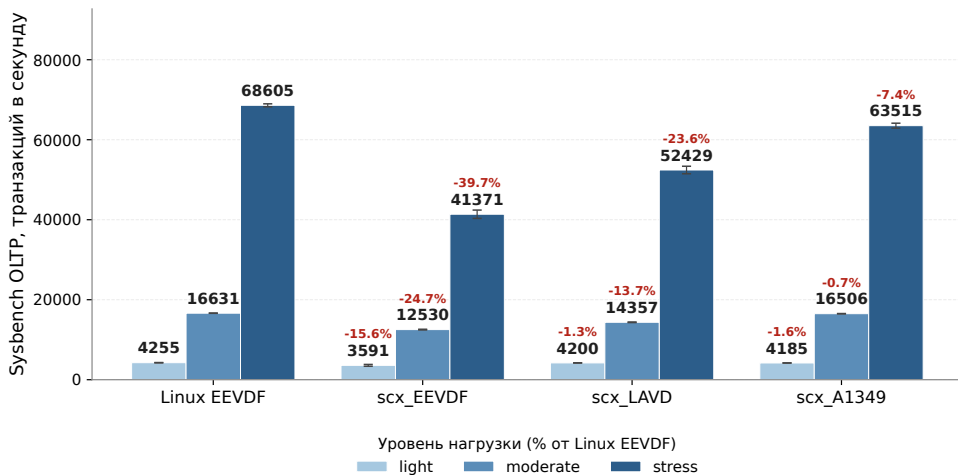


Рис. 3. Результаты sysbench oltp_read_only

schbench: средний RPS (больше — лучше)

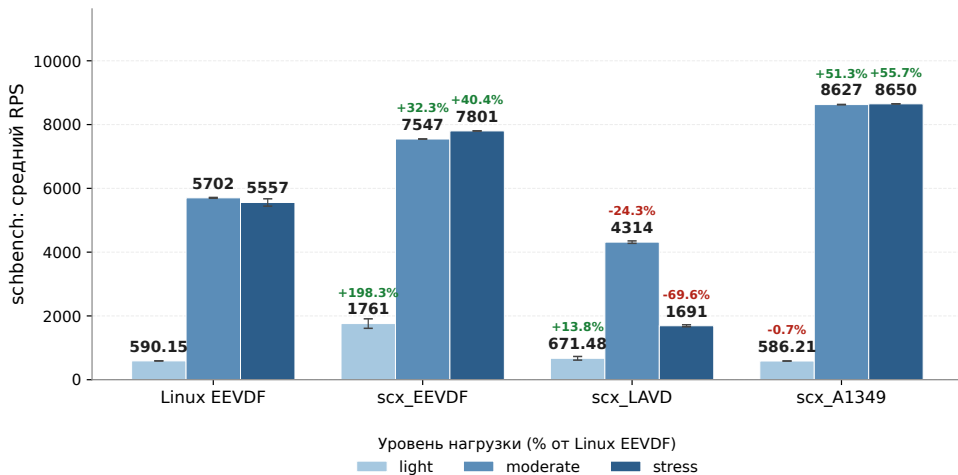


Рис. 4. Результаты schbench

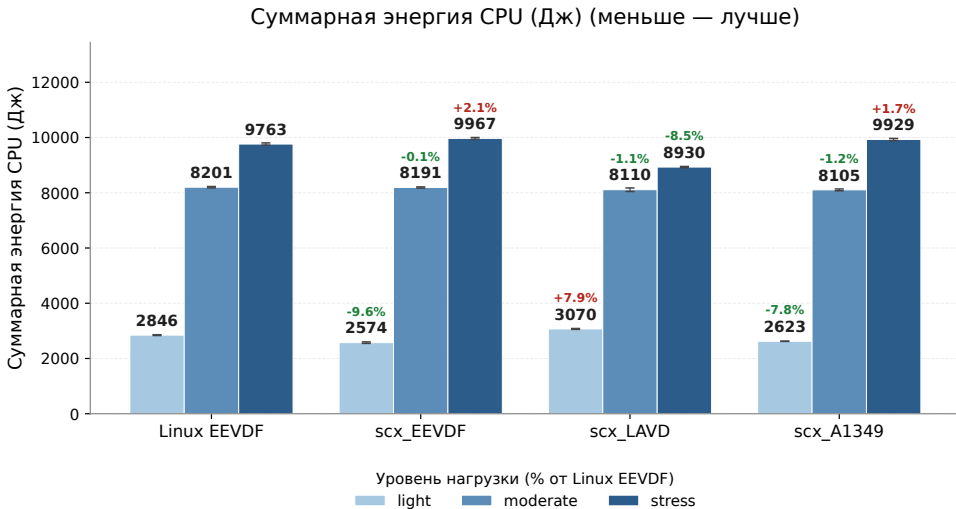


Рис. 5. Энергопотребление процессора за всё время тестирования по счётчикам Intel RAPL

- **hackbench:** scx_A1349 превосходит штатный EEVDF на 4,5 / 8,5 / 1,4 %
- **schbench:** лидерство по средней пропускной способности при умеренной и стресс-нагрузке на 51,3 / 55,7 %. Задержка p99.9 при стресс-нагрузке 24,5 мс против $\approx 1,7$ с у EEVDF
- **sysbench:** единственная нагрузка где scx_A1349 проигрывает EEVDF (1,6 / 0,8 / 7,4 %). Отстают все sched_ext-планировщики до 39,7 %, причиной могут являться накладные расходы фреймворка

- Обоснован выбор `sched_ext` среди четырёх рассмотренных подходов к реализации планировщика
- Построена математическая модель распределения квантов P/E ядер как динамического аукциона VCG
- Реализован планировщик `scx_A1349` на основе построенной модели
- Проведён эксперимент на реальном гетерогенном оборудовании, проведено сравнение со штатным планировщиком Linux и современными eBPF-планировщиками
- По части материалов работы подготовлена статья, прошедшая рецензирование, и сделан доклад на конференции CPS

Спасибо за внимание!

- EEVDF Earliest Eligible Virtual Deadline First — штатный планировщик ядра Linux с 6.6
- sched_ext инфраструктура ядра для подключаемых планировщиков на eBPF
- eBPF extended Berkeley Packet Filter, верифицируемый байт-код, исполняемый в ядре
- ghOSt пользовательский планировщик от Google, политика в user-space
- scx_LAVD Latency-Aware Virtual Deadline, eBPF-планировщик с учётом отзывчивости
- scx_EEVDF порт EEVDF поверх sched_ext, эталон накладных расходов фреймворка
- DSQ Dispatch Queue, очередь готовых задач внутри sched_ext
- VCG Vickrey-Clarke-Groves, аукцион с правдивыми оценками
- MDP Markov Decision Process, марковский процесс принятия решений
- EWMA Exponentially Weighted Moving Average, экспоненциально-взвешенное скользящее среднее
- RAPL Running Average Power Limit, счётчики энергии Intel
- p99 / p99.9 99-й и 99,9-й процентиля задержки
- P / E ядра Performance / Efficient ядра в гибридной архитектуре Intel
- σ коэффициент замедления, отношение пиковой производительности P-ядра к E-ядру

Пример: $K_P = K_E = 1$, одно свободное ядро типа κ

Победитель:

$$i^* = \arg \max_{i \in \mathcal{N}^t, B_i^t > 0} \phi_\kappa(\theta_i) + \delta^{m_\kappa(l_i)} \bar{W}_\kappa$$

где $m_P(l) = l$, $m_E(l) = \lceil l \cdot \sigma \rceil$

Платёж победителя:

$$p_{i^*} = \phi_\kappa(\theta_j) + (\delta^{m_\kappa(l_j)} - \delta^{m_\kappa(l_{i^*})}) \bar{W}_\kappa$$

где j — задача на втором месте, $\bar{W}_\kappa$ — ожидаемое будущее благосостояние кластера.

$\phi_P(\theta_i) > \phi_E(\theta_i)$ выполняется не всегда. При малом v_i и большом l_i E-ядро может дать большую эффективную ценность ($c_E \lceil l_i \sigma \rceil < c_P l_i$ при $\sigma < c_P / c_E$). Оптимальная стратегия нетривиальна.

В модели

агент i
ценность v_i
длина l_i
платёж p_i
аукцион $\arg \max_{\kappa} \phi_{\kappa}$
контракт $m_{\kappa}(l_i)$
бюджет B_i^t

В коде scx_A1349

```
task_struct → auction_task_ctx  
p→scx.weight (weight_cached)  
len_est_ns EWMA  
vcg_payment() списывает токены  
выбор DSQ_P / DSQ_E по  $\phi$   
contract_length(), delta_pow  
tctx→budget, budget_max
```

Что если все `scx.weight` равны?

При $v_i = w = \text{const}$ значение выносится из $\arg \max$, решение определяется только длиной через $m_{\kappa}(l_i)$. Короткие задачи делят ϕ и попадают на P в порядке FIFO внутри DSQ. Длинные задачи через возрастание $m_E = \lceil l\sigma \rceil$ смещаются на E. Патовой ситуации нет. Стратегический мотив агента сводится к выбору nice / cgroup веса.

$\sigma = \eta_P / \eta_E$ из `cpu_capacity`

Userspace-часть `scx_A1349.c` читает `/sys/devices/system/cpu/cpuN/cpu_capacity`, $\eta_P = \max_{cpu}$, $\eta_E = \min_{cpu}$, передаёт в BPF через `global_data`. На тестовой платформе Core Ultra 7 265K значения ядра: $\eta_P = 1024$, $\eta_E = 800$, отсюда $\sigma \approx 1,28$. P-кластер классифицируется по порогу `P_CAP_PCT = 90`.

c_P , c_E физически

c_κ — удельная стоимость кванта на кластере κ в единицах w . По умолчанию $c_P = 1024$. Если c_E не задан, выводится $c_E = c_P \cdot \eta_E / \eta_P$, то есть $\gamma = c_P / c_E = \sigma$. Это удерживает $\phi_P \approx \phi_E$ номинально, смещает выбор за счёт m_E .

Единицы

$v_i = w_i = \text{p} \rightarrow \text{scx.weight}$, безразмерный, 1...10000, по умолчанию 100.

l_i хранится как `len_est_ns` (нс), в кванты $l_q = \text{len_ns} / \text{SLICE_P}$, `SLICE_P = 20` мс.

$B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$.

Extra. Накладные расходы sched_ext на sysbench

Контрольный эксперимент. scx_EEVDF это тот же алгоритм EEVDF, портированный поверх sched_ext. Разрыв штатный EEVDF против scx_EEVDF изолирует накладные расходы фреймворка от логики политики.

	Режим	штатный EEVDF	scx_EEVDF
sysbench oltp_read_only, TPS (выше лучше)	лёгкий	4 255	3 591 (−15,6%)
	умеренный	16 631	12 530 (−24,7%)

Тот же алгоритм, разная среда исполнения. Разрыв в 664 и 4101 TPS это плата за sched_ext.

Чем подтверждается на уровне планировщика

- рост context switch rate под sysbench
- каждое переключение добавляет вход в BPF-программу и обновление DSQ
- стоимость pinned-задачи на одно ядро делает фиксированную стоимость заметной, тогда как haskbench её амортизирует на массе сообщений

scx_A1349 отстаёт от штатного на 1,6% и 0,8%, но опережает scx_EEVDF на 16,5% и 31,7%: учёт гетерогенности компенсирует накладные расходы фреймворка.

Признак	scx_A1349	scx_LAVD	PIE	EEVDF
Что оптимизирует	благосостояние кластера	виртуальный дедлайн с поправкой на latency	пропускная способность гетерогенной системы	справедливость по runtime
Гетерогенность	явно, через σ , $m_E(l) = \lceil l\sigma \rceil$	нет явной модели типа ядра	да, SPI-стек из аппаратных счётчиков	нет
Экон. модель	VCG-аукцион, бюджеты	нет	нет	нет
Решение	MDP, максимум ϕ минус платёж	сортировка по виртуальному дедлайну	предсказание замедления на альт. ядре	eligible-задача с минимальным дедлайном
Ablation	да, ручки на σ , бюджеты, δ	ограниченно	—	—

Ключевое отличие. LAVD оптимизирует одну задачу относительно очереди. PIE предсказывает поведение на альтернативном ядре по счётчикам. scx_A1349 единственный формулирует распределение как механизм со стимулирующей совместимостью на множестве задач и кластеров.

Extra. Валидация scx_EEVDF как эталона

Зачем. scx_EEVDF это эталон накладных расходов sched_ext. Корректность сравнения зависит от того, насколько точно он воспроизводит штатный EEVDF.

Воспроизведено

- vruntime и eligible deadline
- lifecycle ops sched_ext (enqueue, dispatch, running и т.д.)
- выбор eligible-задачи с минимальным дедлайном

Нельзя точно сохранить

- PELT и wakeup affinity (закрытые fair-hooks)
- штатные балансировка и NUMA-миграции
- cgroups bandwidth control (частично)

Эмпирическая проверка. CPU-bound (hackbench): штатный 1,896с vs scx_EEVDF 1,850с (лёгкая); 9,729 vs 8,710 (умеренная). Эквивалентность алгоритма подтверждена.

I/O-bound (sysbench): 4255 → 3591 TPS (−15,6%), 16631 → 12530 TPS (−24,7%). Алгоритм тот же, значит наблюдаемое это стоимость sched_ext на каждом переключении.

Extra. Механизм выигрыша schbench p99.9

Трасса одного запроса schbench: sleep $\rightarrow$ matrix_mult $\rightarrow$ sleep. Метрика *request latency* измеряет фазу matmul, она доминирует в хвосте p99.9.

Штатный EEVDF при стрессе. Фаза matmul конкурирует со всеми CPU-bound задачами за P-ядра единым virtual time. При $Q \gg K_P$ лаг отдельных запросов растёт линейно по числу впереди стоящих, в хвосте $\Rightarrow$ p99.9 $\approx$ 1,69 с.

scx_A1349. Для matmul-фазы l_i велико, $\phi_P(\theta_i) = v_i - c_P l_i$ остаётся выше $\phi_E(\theta_i) = v_i - c_E \lceil l_i \sigma \rceil$, $\arg \max_{\kappa} \phi_{\kappa} \rightarrow$ AUCTION_DSQ_P. Фоновые задачи с малым v_i через тот же $\arg \max$ уходят в AUCTION_DSQ_E. P-кластер обслуживает только matmul-фазы, очередь короткая, p99.9 = 24,5 мс.

EEVDF		P:[mm mm bg bg mm bg ..] один поток vtime
A1349		P:[mm mm mm] E:[bg bg bg] разделение по ϕ

Extra. DSQ_STARVED и защита от голодания

AUCTION_DSQ_STARVED (id = 3) — очередь задач, не прошедших аукцион ($p_i > B_i^t$). Низкий приоритет, обслуживается после DSQ_P и DSQ_E.

Пополнение бюджета. За время сна Δt : $\Delta B = \min(\Delta t, \text{IDLE_CAP}) \cdot w_i / \text{REPLENISH_DIV}$, до потолка $B_i^{\max} = w_i \cdot \text{BUDGET_MUL}$. Задача в STARVED накапливает бюджет до победы в следующем аукционе.

Гарантия прогресса. STARVED обслуживается FIFO по ϕ -vtime, eventual progress даже при пустом бюджете. Голодание невозможно: либо победа после пополнения, либо обслуживание при простое P/E-кластеров.

Инверсия приоритета. Отдельный PI-обработчик в BPF не реализован. Защита наследуется от ядерного futex PI через rt_mutex, sched_ext видит повышенный $p \rightarrow \text{prio}$ и пересчитывает w_i при следующем enqueue.

Кэш-локальность в `select_cpu`. Быстрый путь: `prev_cpu` без сканирования, если простаивает в нужном кластере. Полный путь: `bpf_for` ищет свободное P-ядро. Длинные задачи ($l \geq \text{SLICE_P}$) уходят в per-CPU sticky DSQ `PERCPU_BASE+cpu`, сохраняя L1/L2.

Стоимость миграции. Явного члена нет, входит косвенно через σ : перенос $P \rightarrow E$ удлиняет контракт до $\lceil l_i \sigma \rceil$, снижает ϕ_E и штрафует миграцию для крупных l_i .

Перегрузка P-кластера: $Q_P \geq K_P$ и $Q_P \cdot K_E > Q_E \cdot K_P$. Эквивалентно $Q_P/K_P > Q_E/K_E$ — нормированная глубина очереди P выше E. Перекрёстное умножение вместо деления убирает 64-битный `div` на быстром пути.

Активируется для коротких задач ($\hat{l}_{ns} < \text{SLICE_P}$): даже с σ -замедлением $c_E \lceil l_i \sigma \rceil$ меньше ожидания в очереди P. Trade-off: throughput при перегрузке vs строгая P-affinity.

Платформа

Intel Core Ultra 7 265K (Arrow Lake-S),
8P (Lion Cove) + 12E (Skymont), без SMT,
20 аппаратных потоков
32 ГБ DDR5 6000 МГц, TDP 125 / 250 Вт
Arch Linux, ядро Linux 7.0

Управление частотой и питанием

Регулятор powersave поверх intel_pstate,
HWP активен, EPP по умолчанию
Turbo Boost включён
C-states включены
Микрокод 0x121

Снижение шума измерений

Перед каждым запуском пауза 30 с для сброса частот и термического состояния

Первые 5 с каждой фазы отбрасываются как разогрев

Случайная перестановка порядка планировщиков на каждой итерации

Десктопное окружение и фоновые сервисы зафиксированы между запусками

Границы выводов

Количественные значения

привязаны к данной конфигурации

При смене регулятора, отключении Turbo, иной пропорции P/E или включённом SMT

ожидается сохранение качественного паттерна (преимущество на schbench p99/p99.9), но абсолютные числа изменятся

Дизайн выборки

$N = 6$ независимых запусков на пару (уровень, планировщик)

4 планировщика, 3 уровня нагрузки, $4 \cdot 3 \cdot 6 = 72$ полных запуска

Случайная перестановка порядка планировщиков, межзапусковые паузы по 30 с

Доверительные интервалы

95% интервал по t -распределению Стьюдента, $df = N - 1 = 5$

Реализация `scipy.stats.t.interval`

Bootstrap сознательно не используется: при $N = 6$ ресэмплинг лишь перетасовывает имеющиеся значения и даёт занижено узкий интервал

Почему $N = 6$ достаточно

Системный бенчмарк, не A/B на людях: коэффициент вариации $CV = \sigma/\mu$ мал ($\approx 0,2 - 2\%$ по большинству метрик)

Полуширины 95% интервалов укладываются в единицы процентов от среднего
Опора на ЦПТ для выборочного среднего, не на нормальность сырых наблюдений

Размер эффекта vs шум

scx_A1349 vs EEVDF на стрессе:

schbench RPS +51%,

p99.9 wakeup $\approx 70\times$ ниже

Эффект на порядки больше выборочного σ , интервалы разнесены без перекрытия

Шапиро-Уилка при $N = 6$ малоинформативен, вместо него опора на размер эффекта и t -интервал

Ограничения BPF verifier

Циклы только с доказуемой границей (bpf_loop, bpf_for, развёртка на этапе компиляции)

Рекурсия запрещена

Стек программы ограничен 512 байтами

Лимит инструкций после JIT (порядка 10^6)

Динамическая память общего назначения недоступна, состояние только через BPF maps

Арифметика с плавающей точкой запрещена

Как алгоритм укладывается в эти границы

$\arg \max_{\kappa} \phi_{\kappa}$ берётся по двум кластерам $\{P, E\}$ (две ветви), не по $\mathcal{N}^t$

$\bar{W}_{\kappa}$ как EWMA реализованного ϕ_{κ} , а не точный counterfactual W_{-i} : точный VCG-расчёт за $O(N^2)$ несовместим с лимитом инструкций

l_i оценивается через EWMA len_est_ns

Состояние задач в BPF_MAP_TYPE_TASK_STORAGE, табличные данные в BPF_MAP_TYPE_ARRAY

Бюджет, ϕ и платежи в фиксированной целой арифметике: PHI_BIAS = 2^{62} , DELTA_SCALE = 2^{20}

Это не дефекты реализации, а проектные ограничения eBPF. Математическая модель сохраняет VCG-структуру и инвариант бюджета, упрощения относятся к оценке W_{-i} и l_i .

- [1] Lions J. A Commentary on the Sixth Edition UNIX Operating System. — Sydney: The University of New South Wales, 1977.
- [2] Kravetz M., Franke H., Nagar S., Ravindran R. Enhancing Linux Scheduler Scalability // Proceedings of the Ottawa Linux Symposium. — 2001.
- [3] Love R. Linux Kernel Development. — 2nd ed. — Indianapolis: Novell Press, 2005. — 401 p.
- [4] Wong C. S., Tan I. K. T., Kumari R. D., Lam J. W., Fun W. Fairness and Interactive Performance of O(1) and CFS Linux Kernel Schedulers // International Symposium on Information Technology (ITSim 2008). — Kuala Lumpur: IEEE, 2008. — P. 1–6.
- [5] Stoica I., Abdel-Wahab H. Earliest Eligible Virtual Deadline First: A Flexible and Accurate Mechanism for Proportional Share Resource Allocation: Technical Report TR-95-22. — Norfolk: Old Dominion University, 1995.
- [6] Greenhalgh P. Big.LITTLE Processing with ARM Cortex-A15 & Cortex-A7: ARM White Paper. — Cambridge: ARM Ltd., 2011.
- [7] Rotem E. et al. Alder Lake Architecture // 2021 IEEE Hot Chips 33 Symposium (HCS). — Palo Alto, CA, USA, 2021. — P. 1–23. — DOI: 10.1109/HCS52781.2021.9567097.
- [8] Conti F., Paulin G., Garofalo A., Rossi D., Di Mauro A., Rutishauser G., Ottavi G., Eggimann M., Okuhara H., Benini L. Marsellus: A Heterogeneous RISC-V AI-IoT End-Node SoC with 2-to-8b DNN Acceleration and 30%-Boost Adaptive Body Biasing // IEEE Journal of Solid-State Circuits. — 2024. — Vol. 59, No. 1. — P. 128–142.
- [9] Energy Aware Scheduling [Электронный ресурс] // Linux Kernel Documentation. — URL: <https://www.kernel.org/doc/html/latest/scheduler/sched-energy.html> (дата обращения: 14.11.2025).
- [10] Corbet J. The BPF extensible scheduler class [Электронный ресурс] // LWN.net. — 30.11.2022. — URL: <https://lwn.net/Articles/916291/> (дата обращения: 20.11.2025).
- [11] Corbet J. Sched_ext at LPC 2024 [Электронный ресурс] // LWN.net. — 2024. — URL: <https://lwn.net/Articles/991205/> (дата обращения: 11.09.2025).

- [12] Humphries J. T., Natu N., Chaugule A., Weisse O., Rhoden B., Don J., Rizzo L., Rombakh O., Turner P., Kozyrakis C. gh0St: Fast & Flexible User-Space Delegation of Linux Scheduling // Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP 2021). — New York: ACM, 2021. — P. 588–604.
- [13] Tang Q., Gao X., Li G., Lin J. Optimizing Linux Scheduling Based on Global Runqueue with SCX // IEEE International Conference on Systems, Man, and Cybernetics (SMC 2024). — IEEE, 2024.
- [14] Xu J., Wolff D., Han X. Y., Li J., Roychoudhury A. Concurrency Testing in the Linux Kernel via eBPF: preprint. — arXiv, 2025. — arXiv:2504.21394.
- [15] Mao J., Ujcich B. E., Ma S. Rethinking Provenance Completeness with a Learning-Based Linux Scheduler: preprint. — arXiv, 2025. — arXiv:2510.08479.
- [16] Wang X., Jia S., Huang Z., Cao J., Song M. Mixture-of-Schedulers: An Adaptive Scheduling Agent as a Learned Router for Expert Policies: preprint. — arXiv, 2025. — arXiv:2511.11628.
- [17] Galin M., Hedblom A. Linux Kernel Scheduler Evaluation for Performance-Critical Telecom Workloads: Master's Thesis. — Linköping: Linköping University, 2025. — 74 p.
- [18] Van Craeynest K., Jaleel A., Eeckhout L., Narvaez P., Emer J. S. Scheduling Heterogeneous Multi-Cores through Performance Impact Estimation (PIE) // Proceedings of the 39th Annual International Symposium on Computer Architecture (ISCA 2012). — New York: ACM, 2012. — P. 213–224.
- [19] Sano R. A Dynamic Mechanism Design for Scheduling with Different Use Lengths: KIER Discussion Paper No. 924. — Kyoto: Institute of Economic Research, Kyoto University, 2015.
- [20] Dobzinski S., Lavi R., Nisan N. Multi-unit Auctions with Budget Limits // Games and Economic Behavior. — 2012. — Vol. 74, No. 2. — P. 486–503.